Project Report (40 points)

1. Add all the plots generated as part of this assignment. **This is how we will be primarily grading correctness of your code.** For the 3d plots, just take a screen shot from any angle.

2. In part 1, we saw that the embedding, $\phi((x_1, x_2)) \rightarrow (x_1, x_2, x_1^2 + x_2^2)$, was able to make our non linearly separable data in $\mathbb{R}^2$ perfectly linearly separable in the higher dimensional feature space. Give a non rigorous explanation using the embedding function and the nature of the data regarding why the polar embedding was able to make *this particular dataset* perfectly linearly separable. Would this embedding be able to make any data set linearly separable? Feel free to use the plots generated in part 1 to support your answer
   The data is circular, one class forms a ring around the other. In 2D no straight line can separate them. The polar embedding adds a new feature representing the squared distance from the origin. Since the two classes sit at different distances from the origin, this new feature pulls them apart in 3D where a flat plane can separate them. This would not work for any dataset since it only works when the classes differ in their distance from the origin. Visually it seperates well into a 3D model the graphs match.

3. In part 2, we visually confirmed the solution found by the SVM by manually embedding and training a linear SVM vs training a SVM using a corresponding kernel were the same. In lecture we mathematically proved that they are the same. If this is the case, then why would we use a kernel function rather than manually embedding? Give two reasons why this is the case.
   1.
   Computational efficiency: Manually embedding transforms every data point into a higher dimensional space before computing dot products. A kernel computes the inner product in that space directly from the original data, which is much cheaper, especially when the embedding dimension is very large.
   2.
   Memory: Manually embedding requires storing all data points in the higher dimensional space, which can be enormous. Kernels only require storing the original data and computing pairwise kernel values, saving significant memory.

4. In part 3, what trend do you see regarding the complexity of the decision boundaries as C increases? When would we want to use low C values or high C values?
   As C increases, the decision boundaries become more complex. High C heavily penalizes misclassifications so the boundary contorts to fit the training data, while low C tolerates misclassifications producing a simpler

boundary. Use low C for noisy data to avoid overfitting, and high C for clean data where fitting closely is desirable.

5. In part 4, does the plot of the SVM trained on the entire dataset vs the plot of the SVM trained on only the support vectors match your expectations? Explain why you had those expectations. It's fine if the plots contradicted your expectations.
   The plots match expectations. The decision boundary and margins in the top row (trained on all data) are identical to those in the bottom row (trained on only the support vectors). Support vectors are the only points with non-zero dual coefficients ($\alpha > 0$), meaning non-support vector points contribute nothing to w and therefore nothing to the decision boundary.

6. In part 4, comment on the margins as we remove support vectors. Explain why this is the case using the optimization problem presented in lecture, and the definition of support vectors.
   The margins widen, This is because support vectors are the points closest points to the decision boundary. In the optimization problem, w is a linear combination of only the support vectors, meaning they are the only points actually changing the solution. When we remove them, the next closest points become the new support vectors, but the optimizer is now free to find a wider margin without breaking the contraints.